

INTRODUCCIÓN – Ponerse en tema desde cero

*Tesis: Pronóstico calibrado de entrega agéntica. Maestría en IA, Universidad ORT Uruguay. Para leer antes que nada. No hace falta saber de desarrollo de software ni tener fresca la estadística. Los ejemplos numéricos de este documento son **inventados y calculados acá**, no son datos de Rootstrap. Están para que los conceptos se vean, no para afirmar nada sobre la realidad.*

Cómo usar este documento

Está pensado para leerse una vez de corrido, en un rato. Después queda como referencia: cuando en una reunión alguien diga "PIT" o "sharpness" y no te suene, volvés a la sección correspondiente.

Sección	Qué contesta
1	¿Cuál es el problema y por qué recién ahora se puede atacar?
2	¿Qué construyó Rootstrap y qué hacemos nosotros?
3	Los conceptos de probabilidad, en orden, con ejemplos
4	¿Cómo se le pone nota a un pronóstico? (el corazón de la tesis)
5	Las tres trampas que pueden arruinar el trabajo
6	Qué entregamos y qué no
7	Qué leer, en qué orden
8	Glosario

1. El problema

Por qué estimar software siempre falló

Hace cincuenta años que la industria del software intenta responder "¿cuánto va a tardar?" y sistemáticamente se equivoca. Hubo muchos métodos —puntos de función, COCOMO, story points, planning poker— y todos fracasaron parecido. Las dos razones de fondo:

El proceso era distinto en cada equipo. Una "metodología" es un ritual que cada equipo reinterpreta. Si el proceso cambia en cada proyecto, no hay nada estable que medir: es como intentar calibrar una balanza que se recalibra sola cada vez que la usás.

Los datos eran auto-reportados. Las horas y los avances los cargaba la misma gente evaluada por ellos. Eso no es medición, es declaración.

Encima hay un problema humano documentado: la gente es **sistemáticamente sobreconfiada** al estimar. Un estudio con 13 profesionales estimando 15 tareas reales encontró que los intervalos que ellos declaraban con 90% de confianza contenían el valor real solo el **68%** de las veces (Jørgensen & Sjøberg 2003, leído completo). Y no es que no sepan: hay evidencia de que los gerentes perciben como más competentes a quienes dan rangos más angostos, o sea que el sistema **premia** la falsa precisión.

Qué cambió

Cuando el software lo escribe un sistema de agentes de IA en vez de personas, las dos razones de fondo desaparecen a la vez:

- El proceso deja de ser un ritual y pasa a ser **un programa versionado**: hace exactamente lo mismo cada vez, y cuando cambia, queda registrado qué versión hizo qué.
- Los datos dejan de ser auto-reportados y pasan a ser **telemetría**: cada paso deja timestamp automático — cuándo arrancó una tarea, cuántas veces rebotó en QA, cuándo se le pidió una decisión a un humano y cuánto tardó en responder.

Por primera vez existen las condiciones que la estimación siempre necesitó. **Esa es la oportunidad del proyecto.**

La pregunta de la tesis

¿Se puede pronosticar de forma **calibrada** la entrega de software ejecutado por agentes, y qué aporta cada capa del modelo por encima de baselines simples?

La palabra que carga todo el peso es *calibrada*, y la sección 4 la explica. Adelanto la idea: no alcanza con dar una fecha ni con dar un rango. Hay que poder demostrar que **cuando el sistema dice "80% de probabilidad", pasa el 80% de las veces**. Sin eso, un pronóstico probabilístico es una opinión con un gráfico al lado.

Dónde está nuestro aporte

Motores de pronóstico hay desde hace décadas. Lo que no encontramos publicado, después de revisar unos 100 trabajos, es que alguien haya **evaluado la calibración de pronósticos de entrega de software** — ni con humanos ni con agentes. Existe evaluación de calibración de probabilidades binarias en PRs de agentes, y existe evaluación de calibración de distribuciones en costos de infraestructura pública, pero no el cruce.

Nuestro diferencial no es el motor: es la capa de evaluación. Somos los que le ponen nota, y de forma independiente de quien construyó el motor.

2. Qué hay del otro lado: el sistema de Rootstrap

Rootstrap es la empresa que aporta los datos. Tiene un sistema de entrega —lo llaman *rs-ip*— donde cada proyecto corre el mismo ciclo: planificar, construir, revisión adversarial, merge, convergencia de QA. Todo lo ejecutan agentes bajo controles humanos, contra una especificación completa que ellos llaman *Ground Truth*.

Sobre eso escribieron un whitepaper que propone un método de estimación. Cinco piezas:

1. Una unidad de trabajo estable: la *capability*. No es una página ni un sprint: es una función de negocio que podés cotizar y entregar sola ("registrarse", "checkout con suscripción"). Que la unidad sea estable es lo que hace comparables las mediciones entre proyectos.

2. Diez clases de referencia, fijadas antes de ejecutar. Cada *capability* se clasifica en una grilla de 5 niveles de complejidad (C1 a C5) × 2 tamaños (S y L). La clasificación la hace una persona en el mismo momento en que aprueba la especificación, **antes de saber cuánto va a tardar**, y no se toca después. Cada celda arranca con un valor de referencia (una C1-S: un día hábil) que se va corrigiendo con los datos reales.

3. Un modelo de la duración con variables medidas, no preguntadas. Cuántas tareas corren en paralelo, cuánto rebota en QA, qué tan completa está la especificación, cuánto tarda el cliente en contestar una pregunta, más sacudones raros y grandes.

4. Una simulación. Como el proyecto es un grafo de tareas con dependencias, y las duraciones son variables aleatorias, no hay fórmula cerrada. Entonces se juega el proyecto 10.000 veces (sección 3.4).

5. Un protocolo de validación. Cobertura, PIT, CRPS, backtesting. Está escrito en el whitepaper, pero **no está ejecutado**: el propio documento cierra diciendo que el boletín de calibración, y no el paper, es la evidencia de que las promesas se sostienen.

Ahí entra la tesis. Nosotros no construimos ese motor ni lo replicamos. Construimos **la evidencia que falta**, y la construimos desde afuera, que es lo que le da credibilidad.

Una advertencia de vocabulario que confunde: en el vocabulario de Rootstrap, *Ground Truth* **no** significa lo que significa en machine learning. Para ellos es la especificación completa y autoritativa del software a construir.

3. Los conceptos, en orden

Cada uno se apoya en el anterior. Se pueden leer de corrido.

3.1 Una duración no es un número, es una distribución

Si preguntás "¿cuánto tarda una tarea C2-S?", la respuesta honesta no es "3 días". Es: *a veces 2, normalmente 3 o 4, a veces 9, y muy de vez en cuando 20*. Eso —el conjunto de valores posibles con sus probabilidades— es una **distribución**.

Las duraciones tienen tres propiedades que se repiten en todos lados:

- **Tienen piso y no tienen techo.** Nada tarda menos que cero. Pero cualquier cosa puede tardar el triple de lo esperado.
- **Son asimétricas** (con "cola a la derecha"). La distribución normal, la campana simétrica, **no sirve** acá: te diría que tardar 3 días menos de lo esperado es tan probable como tardar 3 días más, y eso es falso.
- **La cola importa mucho más de lo que parece.** Los proyectos no se atrasan por el promedio; se atrasan por las pocas tareas que explotan.

3.2 Cuantiles: la forma útil de hablar de una distribución

Un **cuantil** es el valor por debajo del cual cae un cierto porcentaje de los casos.

- **P50** (la mediana): la mitad de las veces termina antes, la mitad después. Es la fecha "moneda al aire".
- **P80**: 8 de cada 10 veces termina en esa fecha o antes.
- **P95**: pasarse de esa fecha es un evento de 1 en 20.

Y al revés, que es como lo pregunta un cliente: "*¿qué probabilidad hay de estar en producción el 1 de junio?*" — se lee directo de la curva.

La regla que ordena todo: nunca se entrega una fecha sola. Se entrega la curva, y se leen tres números con una frase cada uno. P50 es la fecha a la que se planifica; P80 la fecha a la que se compromete; P95 la fecha a la que se le pone precio al riesgo.

3.3 Por qué no alcanza con sumar

Intuición equivocada: "si tengo 12 tareas y cada una tarda 4 días en promedio, el proyecto tarda 48 días". Está mal por tres razones que se acumulan:

1. **Hay dependencias.** Algunas tareas esperan a otras; otras corren en paralelo.
2. **Hay recursos limitados.** Si hay 2 tracks de trabajo, no podés correr 5 cosas a la vez.
3. **Sumar distribuciones asimétricas no da la suma de los promedios**, y menos aún cuando el resultado depende de *cuál* de los caminos del grafo terminó último.

No hay fórmula cerrada para esto. Por eso se simula.

3.4 Monte Carlo: en vez de resolver el proyecto, jugarlo

La idea es asombrosamente simple, y ese es su mérito:

En lugar de calcular cómo se combinan las distribuciones, **jugás el proyecto entero al azar 10.000 veces** y mirás cómo salió.

Una corrida ("trial") es:

1. Para cada tarea, sortear una duración de la distribución de su categoría.
2. Sortear los rebotes de QA, las esperas por decisiones del cliente, y con baja probabilidad, algún sacudón grande.
3. Armar el cronograma respetando dependencias y tracks disponibles.
4. Anotar la fecha de fin.

Repetís 10.000 veces y tenés 10.000 fechas de fin posibles. Las ordenás: la número 5.000 es el P50, la 8.000 el P80, la 9.500 el P95. **Los cuantiles se leen directamente de la lista ordenada, sin ninguna fórmula.**

Un ejemplo con números reales

Simulé un proyecto de juguete: 12 capabilities con dependencias, categorías con las duraciones típicas del whitepaper, 20.000 corridas. Los números salen de correr el código, no están inventados a mano:

Tracks en paralelo	P50	P80	P95	La corrida más rápida de las 20.000
1	51,8 d	64,3 d	82,2 d	23,0 d
2	43,1 d	55,6 d	73,4 d	16,0 d
3	37,0 d	48,4 d	65,6 d	13,8 d
4	33,9 d	45,2 d	62,6 d	12,7 d

Tracks en paralelo	P50	P80	P95	La corrida más rápida de las 20.000
Ilimitados	31,4 d	42,5 d	59,6 d	—

Tres cosas que se leen ahí y que ninguna estimación puntual te da:

El P80 está unos 12 días por encima del P50. Ese hueco es la incertidumbre. Un proyecto que promete el P50 va a llegar tarde la mitad de las veces, y eso no es mala suerte: es lo que el P50 significa.

Los rendimientos decrecientes son visibles y cuantificados. Pasar de 1 a 2 tracks ahorra 8,7 días de P50. De 3 a 4, solo 3,1. Y con tracks ilimitados el P50 no baja de 31,4 días, porque **la cadena más larga de dependencias es un piso que ninguna cantidad de gente puede cruzar.** Poder decirle a un cliente "el cuarto track te compra tres días" en vez de discutirlo de memoria es media conversación comercial resuelta.

Se puede responder la pregunta al revés. ¿Probabilidad de terminar en 55 días o menos? Con 2 tracks, **79,2%**. Con 3, **88,8%**. Es contar cuántas de las 20.000 corridas terminaron a tiempo.

3.5 Clases de referencia: la "mirada de afuera"

Para simular hay que saber, para cada tarea, de qué distribución sortear. ¿De dónde sale?

Dos maneras de estimar una tarea nueva:

- **Desde adentro:** pensar en esta tarea en particular, imaginar los pasos, sumar. Es lo natural y es lo que sesga: uno imagina el camino sin obstáculos.
- **Desde afuera:** preguntarse *"¿a qué se parece esto, y cuánto tardaron las cosas parecidas?"*, y usar la distribución de esas.

La segunda se llama **reference class forecasting** y viene de Kahneman y Tversky vía Flyvbjerg. La grilla de 10 celdas de Rootstrap es exactamente eso: cada celda es una clase de referencia, y una tarea nueva se pronostica con lo que midió su clase.

El problema no resuelto de este enfoque —y es nuestro problema— es cómo se define la clase. Una revisión sistemática reciente de 61 artículos (Cantarelli et al. 2025, leído completo) lo pone como el punto abierto más serio, y hay evidencia medida del trade-off: cuantas más propiedades exigís para que dos proyectos sean "parecidos", más precisa es la comparación **y menos casos comparables te quedan.** Clase ancha, mucha varianza; clase angosta, poca muestra.

Con 100–300 tareas en 10 celdas, el promedio da 10–30 por celda, pero repartidas de forma despareja va a haber celdas con 3 o 4 casos. Con 4 observaciones, estimar el percentil 95 es esencialmente reportar el máximo que viste.

La salida es el **partial pooling**. Idea: si una celda tiene pocos datos, no la estimás sola ni la fusionás con el resto; la estimás con **una mezcla entre lo que dice su propia evidencia y lo que dicen las celdas parecidas**, y el peso depende de cuántos datos tenga. Una celda con 40 observaciones se manda casi sola; una con 3 se apoya fuerte en sus vecinas. Es exactamente lo que hace un abogado cuando tiene tres casos propios y consulta la jurisprudencia general: no ignora sus tres casos, pero tampoco los toma como toda la verdad. En estadística se llama *shrinkage* o modelo jerárquico, y **es la respuesta técnica a la crítica de la sección anterior**. Ese argumento hay que escribirlo así en la memoria.

3.6 No estacionariedad: el pasado describe un proceso que ya no existe

Si el sistema de agentes mejora —cambia el modelo, cambia el prompt, cambia el loop— entonces las tareas de hace ocho meses las hizo **una máquina distinta** de la que va a hacer las próximas. Ese historial sigue siendo información, pero ya no describe el proceso actual. A eso se le llama **no estacionariedad** (o *concept drift*).

La pregunta operativa: **¿cuánta historia conviene usar?** Toda (más datos, pero mezclás procesos distintos) o solo la reciente (más representativa, pero menos datos). Las respuestas conocidas:

- **Ventana móvil:** usar solo las últimas N tareas.
- **Ponderación por recencia:** usar todo, pero que lo viejo pese menos, con un decaimiento.
- **Ponderación adaptativa:** mantener varios modelos entrenados en distintas épocas y darle más peso al que viene acertando ahora.

Lo que sabemos de la literatura, y no es lo que uno esperaría: **no está para nada claro que las ventanas móviles ayuden**. Hay réplicas con resultado negativo y otras con resultado positivo, según el dataset. Y en el trabajo más sofisticado de esta línea (Minku & Yao 2017, leído completo), al descomponer el método en sus partes, la ponderación dinámica es lo que aporta y el filtro por similitud casi no agrega nada.

Consecuencia para nosotros: OE5(b) no se puede plantear como "confirmar que las ventanas ayudan". Se plantea como "medir cuánta historia conviene en este proceso", con la hipótesis nula de que no importa. Que no importe también sería un resultado.

4. El corazón: ¿cómo se le pone nota a un pronóstico?

Esta sección es la tesis. Si de todo el documento te queda una sola cosa, que sea esta.

4.1 El problema de fondo

Un pronóstico puntual se evalúa fácil: dijiste 40 días, tardó 47, erraste por 7. Pero **un pronóstico probabilístico no se puede evaluar con una sola observación**. Si el sistema dijo "P80 = 55 días" y tardó 60, ¿se equivocó? No necesariamente: 2 de cada 10 veces *tiene* que pasarse. Ese es el significado del 80%.

Un pronóstico probabilístico solo se puede juzgar **sobre muchos pronósticos a la vez**. Ese es el motivo de que la unidad de análisis del proyecto sea la tarea y no el proyecto: con 10 proyectos no se puede evaluar nada, con 100–300 tareas sí.

4.2 Calibración: que los números signifiquen lo que dicen

Calibración es la propiedad de que las probabilidades declaradas coincidan con las frecuencias observadas. De todas las veces que dijiste "P80", el resultado real tiene que haber caído en esa fecha o antes en aproximadamente el 80% de los casos.

Se mide contando. Es aritmética de primaria: mirás todos los pronósticos históricos, contás cuántos cayeron por debajo del P80 declarado, dividís. Si da 80%, calibrado. Si da 60%, el sistema es sobreconfiado (promete más precisión de la que tiene). Si da 95%, es demasiado conservador.

4.3 PIT: el diagnóstico completo, no solo un punto

La cobertura mira un cuantil por vez. El **PIT** (*probability integral transform*) mira la distribución entera de una sola vez, y la idea es más simple que el nombre:

Para cada tarea terminada, preguntá: **¿en qué percentil de mi distribución pronosticada cayó el valor real?**

Si el pronóstico era bueno, esos percentiles deberían repartirse **parejo** entre 0 y 100. Igual de frecuente que un resultado caiga en el percentil 10 que en el 60 o en el 90. Entonces hacés un histograma de esos valores y **lo que buscás es que sea chato**. Y la forma que tiene cuando no es chato te dice exactamente qué está mal:

Forma del histograma	Qué significa
Chato	Bien calibrado
Forma de U (mucho en los extremos)	Distribuciones demasiado angostas — exceso de confianza; la realidad se te escapa por las colas
Joroba en el medio	Distribuciones demasiado anchas — te curás en salud, los intervalos son inútiles de tan grandes

Forma del histograma	Qué significa
Rampa (inclinado a un lado)	Sesgo — tu mediana está corrida

Con números

Simulé 300 tareas cuya duración real es lognormal con mediana 4 días, y tres pronosticadores que **aciertan la mediana** pero difieren en cuánta incertidumbre declaran:

Pronosticador	Cobertura P50	Cobertura P80	Cobertura P95	Ancho P50→P95	CRPS
Honesto	47,7%	83,3%	96,0%	8,7 d	1,84
Exceso de confianza	47,7%	65,0%	82,7%	3,1 d	1,99
Vago	47,7%	95,3%	100,0%	29,9 d	2,23

Y sus histogramas PIT, en 10 bins (lo esperado es 10,0% en cada uno):

Honesto	10.3	6.0	10.7	10.3	10.3	8.0	12.7	15.0	8.3	8.3	→ chato
Exceso de confianza	24.3	8.3	4.7	6.0	4.3	4.0	4.3	9.0	12.0	23.0	→ U
Vago	0.0	4.7	9.7	15.7	17.7	19.7	18.7	9.3	4.3	0.3	→ joroba

Las formas salen tal cual las predice el libro. El sobreconfiado tiene **24% de los casos en cada extremo** cuando debería tener 10%: la realidad se le escapa por arriba y por abajo mucho más de lo que su modelo admite. Y su "P80" en realidad cubre el 65%.

4.4 Sharpness, y por qué la calibración sola no alcanza

Acá está la sutileza que más importa, y la que más cuesta ver.

Es trivial estar perfectamente calibrado y ser completamente inútil. Si digo "el proyecto va a terminar entre mañana y dentro de tres años, con 95% de confianza", voy a acertar el 95% de las veces. Calibración impecable, valor cero.

Por eso hace falta una segunda propiedad: **sharpness** (agudeza), que es qué tan concentrada está la distribución. Y el orden entre las dos no es negociable:

Primero calibración, después sharpness. Entre los pronósticos calibrados, gana el más angosto. Un rango angosto sin calibración no es precisión: es precisión falsa.

Esto no es una opinión metodológica nuestra: es el paradigma de Gneiting, Balabdaoui y Raftery (2007), que es la referencia canónica del área. Y ellos demuestran algo que **cambia el diseño de nuestro torneo**: construyeron cuatro pronosticadores de calidad muy distinta que producen coberturas prácticamente idénticas (51,2% / 51,3% / 50,1% / 50,9% contra un nominal de 50%) e histogramas PIT igual de chatos. **La cobertura y el PIT no los distinguen**. Lo que los separa es el ancho y el CRPS.

Por qué esto es exactamente nuestro problema

Uno de nuestros baselines es el **conteo puro**: tratar todas las tareas como intercambiables, sin categorías, y usar la distribución global. Simulé eso: 600 tareas de tres categorías con medianas distintas (1,5 / 4 / 10 días), y dos pronosticadores — uno que condiciona en la categoría y uno "climatológico" que usa la distribución de todo junto:

	Cob. P50	Cob. P80	Cob. P95	Ancho P50→P95	CRPS
Condicionado en la categoría	56,3%	84,0%	95,8%	6,8 d	1,41
Climatológico (todo junto)	49,2%	81,2%	96,7%	13,0 d	2,31

PIT en 10 bins:

Condicionado	10.2	11.3	9.5	11.0	14.3	9.3	8.8	9.5	7.7	8.3
Climatológico	9.7	11.5	8.8	9.2	10.0	10.2	9.7	12.2	11.2	7.7

Los dos están calibrados. El climatológico tiene el PIT incluso más chato. Y sin embargo sus intervalos son **el doble de anchos** y su CRPS es 64% peor. Un cliente al que le decís "entre 2 y 15 días" no recibió información.

Consecuencia directa para el diseño de R4: si evaluáramos el torneo solo por cobertura, el baseline tonto empataría con el modelo, y concluiríamos que las categorías no sirven. Sería una conclusión falsa producida por una métrica mal elegida. El torneo tiene que rankear por CRPS y por ancho, **con la calibración como filtro previo, no como criterio de ranking**. Esto hay que dejarlo escrito antes de correr nada.

4.5 CRPS: un solo número que combina las dos cosas

Cobertura y ancho son dos números y a veces se contradicen. El **CRPS** los resume en uno solo.

Sin fórmula, la intuición: mide **cuánta área hay entre la curva que pronosticaste y lo que efectivamente pasó**. Penaliza dos cosas a la vez — estar centrado en el lugar equivocado, y ser innecesariamente ancho. Tiene tres propiedades que lo hacen la métrica del torneo:

- **Es "propio"**: no se puede hacer trampa. No hay forma de mejorar tu CRPS declarando una distribución distinta de la que realmente creés. Un pronosticador que se cura en salud pone rangos anchos y el CRPS lo castiga.
- **Está en las mismas unidades que el dato**: si medís en días, el CRPS está en días.
- **Si tu pronóstico es un solo número, el CRPS se reduce al error absoluto**. Por eso podés comparar directamente un modelo probabilístico contra un baseline que da una fecha sola. Es lo que hace posible el torneo.

(Existe también la "pérdida pinball", que es lo mismo pero cuantil por cuantil; sirve para ver en qué parte de la curva estás perdiendo.)

4.6 Backtesting con origen móvil

Falta responder **con qué datos** se calcula todo lo anterior. No se puede evaluar un modelo con los mismos datos con los que se entrenó: eso mide memoria, no capacidad de pronóstico.

La forma correcta cuando hay tiempo de por medio:

Parate en un momento del pasado. Usá **solo** lo que existía hasta ahí. Pronosticá lo que vino después. Anotá el resultado. Movete un paso adelante. Repetí.

Se llama **backtesting con origen móvil** (*rolling origin*), y simula honestamente lo que hubiera pasado si el sistema hubiera estado corriendo en ese momento.

Lo tentador es usar validación cruzada normal —barajar todo y separar al azar— porque da más datos y es lo que uno hace en machine learning. **Y da resultados sistemáticamente mejores, que es justamente el problema**. Sigweni, Shepperd y Turchi (2016, leído completo) lo midieron sobre dos datasets: la validación cronológica da errores **13,7% más altos** en uno y **48,1% más altos** en el otro. Y el dato más importante: en uno de los dos, la interacción fue significativa ($p = 0,046$), lo que significa que **el esquema de validación puede cambiar qué modelo parece el mejor**, no solo el nivel de error.

O sea: si validáramos mal, no solo reportaríamos números demasiado buenos. Podríamos elegir el modelo equivocado.

5. Las tres trampas

5.1 Leakage: usar información del futuro sin darse cuenta

Es el error más común, el más difícil de detectar, y el que invalida todo el trabajo si aparece.

Leakage es que el modelo use, para pronosticar, información que en el momento del pronóstico no existía. Casi nunca es obvio. Formas en que se cuela:

- Entrenar con tareas que terminaron **después** del momento del pronóstico.
- Usar la categoría de una tarea si esa categoría fue asignada **después** de ver cuánto tardó (ver 5.2).
- Normalizar o estandarizar los datos usando el promedio de **todo** el histórico, incluido el futuro.
- Elegir hiperparámetros mirando el resultado del período de test.
- Con tareas del mismo proyecto: separar al azar hace que tareas "hermanas" queden a los dos lados del corte y se filtren entre sí.

La regla, y conviene tenerla escrita en la pared: en cada paso del backtesting, el modelo solo puede usar información que existía en ese momento. Y "ese momento" es el **momento del pronóstico**, no el momento en que la tarea terminó.

Vale saber que los papers que leímos tienen leakage residual y lo admiten: Jørgensen & Sjøberg usaron validación cruzada en vez del orden temporal real y lo declaran; Sigweni et al. ordenan por fecha de *finalización*, con lo cual una tarea entra al entrenamiento cuando termina aunque el pronóstico se hiciera al inicio, y lo listan como trabajo futuro. **Nosotros podemos hacerlo más estricto que ellos, y decirlo es un aporte metodológico gratis.**

5.2 Sesgo retrospectivo en el etiquetado

Todo el modelo condiciona en la categoría de la tarea. Si esa categoría se asigna **después** de saber cuánto tardó, el sistema es una tautología: las tareas largas se etiquetan "complejas" y después el modelo "descubre" que las complejas tardan más.

Hay evidencia experimental clásica de que saber el resultado hace que la gente sobreestime cuán predecible era, y que **no puede corregirlo aunque se lo pidan**. No es mala fe: es cómo funciona la cabeza.

Por eso: el histórico se etiqueta **a ciegas** —solo la especificación, nunca el resultado— y por 2 o más personas independientes. Y como eso hay que hacerlo igual, se aprovecha para medir si dos personas mirando la misma spec coinciden. Ese es el estudio OE5(a), y si el acuerdo es bajo, **eso ya es un hallazgo sobre la taxonomía**, independiente del pronóstico.

*(Detalle técnico que hay que resolver antes de correrlo: la métrica estándar de acuerdo, kappa, tiene umbrales de referencia para el dominio de software (mediana 0,62; apuntar a $\geq 0,63$) pero derivados para categorías **sin orden**. Nuestros tiers C1–C5 **sí** tienen orden — confundir C1 con C2 es menos grave que confundir C1 con C5. Hay que decidir y declarar de*

antemano si tratamos la escala como ordenada o no, porque los umbrales no son los mismos.)

5.3 El alcance que se estira

Somos tres, con dedicación parcial, y seis meses. El anteproyecto separa **comprometido** (R1–R5) de **deseable** (R6–R8) justamente por esto.

La regla de trabajo: **toda propuesta que suma alcance tiene que decir qué sale a cambio**. Y hay un punto de control formal en diciembre de 2026: si los datos no sostienen el torneo a nivel tarea, se recorta el alcance y se declara. Está escrito en el anteproyecto y es el seguro del equipo.

6. Qué entregamos

Comprometido:

Qué es	
R1	Esquema canónico de telemetría y pipeline de ingesta, sobre datos reales
R2	Motor de pronóstico v1 que produce distribuciones y cuantiles
R3	Boletín de calibración automático sobre el histórico, con backtesting de origen móvil
R4	Comparación contra ≥ 2 baselines, con el diseño congelado antes de ver resultados
R5	Informe final con los hallazgos, cualquiera sea su dirección

Deseable: horas de intervención humana como segunda variable pronosticada (R6), el segundo estudio de OE5 (R7), proyección viva sobre un proyecto en curso (R8).

Los baselines del torneo, pre-especificados:

1. **Conteo puro** — todas las tareas intercambiables, sin categorías. *Es el más difícil de vencer y el que hay que reportar sí o sí*. Tiene pedigrí publicado: es exactamente el método de Miranda et al. (2021), que sobre 10 proyectos reales dio un error puntual de ~32% en fechas — y que emitía intervalos al 95% que, en sus propias tablas, cubrieron el real 11 veces de 15. Y como vimos en 4.4, va a estar bien calibrado o casi: lo que hay que mostrar es que es demasiado ancho.
2. **Priors por categoría sin aprendizaje** — la tabla de referencia sin actualización bayesiana. Tiene respaldo publicado: es esencialmente el método de Jørgensen &

Sjøberg (2003).

3. **Estimación humana**, donde haya registro.

Y la regla que hace que esto valga como ciencia:

Un resultado desfavorable es un resultado. Si el modelo está mal calibrado, o si los baselines simples le empatan al modelo completo, **eso es un hallazgo publicable y contraintuitivo, no un fracaso**. El diseño está armado para dar información en cualquier dirección. Hay precedentes publicados de exactamente eso: réplicas con resultado negativo, y estudios donde nada moderno le gana al método clásico.

Por eso el diseño de la evaluación **se escribe primero y se corre después**. Si primero mirás los resultados y después elegís la métrica, encontrás lo que quieras.

7. Qué leer, en qué orden

Todo lo que sigue está en `LITERATURA.md`, con estado de acceso y ficha para los que se pudieron leer completos.

Si vas a leer un solo paper: Gneiting, Balabdaoui & Raftery (2007). Es la fuente del paradigma calibración/sharpness, del PIT y de la demostración de que la cobertura sola no rankea. Está abierto y la ficha resume lo esencial.

Si vas a leer tres: sumá Miranda et al. (2021) —que es nuestro baseline 1, nueve páginas, fácil de leer, y ya está en la carpeta— y Jørgensen & Sjøberg (2003) —que es nuestro baseline 2, y está abierto—. Sigweni et al. (2016), que es corto y justifica el backtesting, va cuarto.

Después, por bloque:

Para entender...	Leé
Qué evaluamos y cómo	Gneiting et al. 2007 → Jørgensen 2019 → Sigweni et al. 2016
Contra qué comparamos	Jørgensen & Sjøberg 2003 → Batselier & Vanhoucke 2016 → Miranda et al. 2021
Cómo se arman y validan las clases	Cantarelli et al. 2025 → El Emam 1999
Cuánta historia usar	Lokan & Mendes 2017 → Minku & Yao 2017

Para entender...	Leé
Dónde está el hueco y el plan B	Kumar et al. 2025 → AIDev 2026 → Dao et al. 2026

Ojo: tres de los papers prioritarios no se consiguieron todavía (Jørgensen 2019, Batselier & Vanhoucke, Lokan & Mendes 2017). [LITERATURA.md](#) §3 tiene la vía concreta para cada uno; el de Jørgensen 2019 está a un clic en un repositorio noruego. Miranda et al. ya está en [tesis/bibliografia/](#).

8. Glosario

Backtesting con origen móvil — Entrenar con lo anterior a un punto, pronosticar lo siguiente, avanzar el punto y repetir. Nunca usar información posterior al origen.

Calibración — Que las probabilidades correspondan a frecuencias reales: de todo lo declarado P80, ~80% debe caer antes de esa fecha.

Capability / tarea — Unidad de trabajo del delivery system: una función de negocio que se puede cotizar y entregar sola. Es nuestra unidad primaria de análisis.

Cobertura de cuantiles — Contar en qué porcentaje de los casos el resultado real cayó antes de cada cuantil declarado.

Concept drift — Ver *no estacionariedad*.

CRPS — Puntaje único de toda la distribución: penaliza estar lejos y ser innecesariamente ancho. Está en las mismas unidades que el dato y se reduce al error absoluto si el pronóstico es puntual, lo que permite el torneo entre modelos.

Cuantil — El valor por debajo del cual cae cierto porcentaje de los casos. P80 = el valor que no se supera 8 de cada 10 veces.

Distribución — El conjunto de valores posibles con sus probabilidades. Lo que se pronostica, en vez de un número.

Ground Truth — En el vocabulario de Rootstrap, **no** es el sentido de machine learning: es la especificación completa y autoritativa del software a construir.

Leakage — Usar, para pronosticar, información que en el momento del pronóstico no existía. El error más común y más difícil de detectar.

Lognormal — Una distribución asimétrica con piso en cero y cola a la derecha; sale de suponer que el *logaritmo* de la duración se distribuye normal. Es la familia habitual para duraciones.

Monte Carlo — Simular el proyecto miles de veces sorteando cada duración al azar, y leer los cuantiles de los resultados ordenados.

No estacionariedad — El proceso cambia con el tiempo, así que el historial viejo describe un proceso que ya no existe.

Partial pooling / shrinkage — Estimar cada celda con una mezcla entre su propia evidencia y la de las celdas parecidas, con peso según cuántos datos tenga. La respuesta al problema de las clases con pocos casos.

PIT — En qué percentil de la distribución cayó cada resultado real. Deberían repartirse parejo: histograma chato = calibrado; U = demasiado angosto; joroba = demasiado ancho; rampa = sesgado.

Regla de puntuación propia — Una métrica que no se puede engañar: no conviene declarar una distribución distinta de la que realmente creés. CRPS y pinball lo son; MMRE no.

Reference class forecasting — Pronosticar una tarea nueva con la distribución de las tareas parecidas ya observadas ("mirada de afuera"), en vez de razonar desde los detalles de esta tarea ("mirada de adentro").

Sharpness — Qué tan angosta es la distribución. Se evalúa **sujeto a** calibración: un rango angosto sin calibración es precisión falsa.

Apéndice: reproducir los números

Los tres ejemplos numéricos salen de scripts cortos que están junto a este documento (`ejemplo.py`, `calibracion.py`, `climatologico.py`). Usan solo numpy y scipy, corren en segundos, y tienen semilla fija. Se pueden trastear para ver, por ejemplo, qué le pasa al PIT si el modelo acierta la dispersión pero yerra la mediana. Recomendando hacerlo: se entiende más en cinco minutos de jugar que en cinco páginas de leer.